

as a singleton, along with the name of the static method for creating the objects?

```
@Singleton(getter = "getInstance")
public class InMemoryUserRepository implements UserRepository {
    ...
}
```

It both improves the code readability and gives the flexibility to supply the name of the static method. The annotation is defined as follows:

```
package com.glarimy.framework;

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface Singleton {
    public String getter();
}
```

Notice that this annotation also has nothing to do with the UMS. So, naturally, it fits into the framework.

With the introduction of the annotation, the *ObjectFactory* needs an improvement to check for the presence of the annotation before falling back on the presence of the *getInstance()* method.

```
Singleton singleton = claz.getAnnotation(Singleton.class);
if (singleton != null) {
    String getter = singleton.getter();
    return claz.getMethod(getter).invoke(claz);
}
return claz.getMethod("getInstance").invoke(claz);
```

Factory method

Though the way we implemented the factory pattern is good, it makes even more sense to make it open for extension. In other words, design a contract for the factory and leave it open for several possible implementations. Here is the contract (i.e., interface):

```
package com.glarimy.framework;

public interface Factory {
    public Object get(String key) throws Exception;
}
```

Here is an implementation for the *Factory* interface. We replaced the static method with an instance method. Also, the constructor accepts the name of the configuration file, instead of hard-coding it.

```
public class ObjectFactory implements Factory {
    private Properties props;
    public ObjectFactory(String source) throws Exception {
        props = new Properties();
        props.load(new FileReader(source));
    }

    @Override
    public Object get(String key) throws Exception {
        ...
    }
}
```

With these changes in the framework, the *Application* looks like this:

```
Factory factory = new ObjectFactory("config.properties");
UserRepository repo = (UserRepository) factory.get("user.
repository");
```

Figure 2 depicts the UMS that we have designed so far.

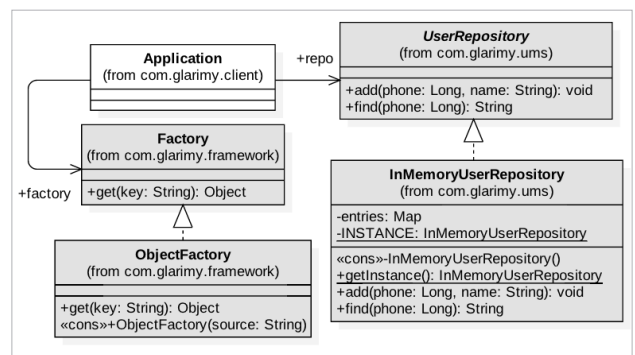


Figure 2: Refactored design for user management system

In this first part of the 12-part series, we covered best practices like having contracts, coding against interfaces, and coupling the consumers and providers loosely. We also understood the intentions behind singleton, factory, and factory method patterns, and found that there is more than one way of implementing them. Since these patterns are related to creating objects, they are categorised as ‘creational patterns’ by the Gang of Four. In the next part, we will be looking at a couple of ‘structural patterns’. **END** 🐧

By: Krishna Mohan Koyya

The author is the founder and the principal technology consultant at Glarimy Technology Services, Bengaluru. He has delivered more than 500 programs to corporate clients on design and architecture in the last 12 years. Prior to that, he held various positions in the industry and academia for a decade.